

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 01 -

FUNDAMENTOS DE ENGENHARIA DE SOFTWARE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	34
O QUE VOCÊ VIU NESTA AULA?	35
REFERÊNCIAS.....	36

EMSE

O QUE VEM POR AÍ?

Imagine que você terminou uma análise de dados brilhante em um notebook Jupyter, mas quando um colega tenta reproduzir o resultado ou implementar em produção, nada funciona direito. Frustrante, não? Situações assim são comuns: estima-se que mais de 80% dos projetos de ciência de dados falham ou nunca chegam a gerar valor de negócio, em grande parte por problemas na transição de um código “de laboratório” para um sistema robusto em produção.

Isso acontece porque sucesso em ciência de dados não é só construir bons modelos – é preciso escrever código de qualidade, organizado e sustentável, que outras pessoas (e você no futuro) consigam executar e manter facilmente. É aí que entram os princípios de Engenharia de Software: eles fornecem disciplina e boas práticas para transformar protótipos experimentais em produtos confiáveis.

Nesta aula, vamos refletir sobre como aplicar engenharia de software em projetos de ciência de dados pode aumentar drasticamente as chances de sucesso de suas iniciativas. Você verá por que um código modular, reproduzível e bem documentado torna seu trabalho mais robusto e escalável – e como a falta dessas práticas pode levar a verdadeiros desastres (como um caso famoso em que milhares de casos de COVID-19 ficaram fora das estatísticas devido a um erro de planilha!).

Também discutiremos o ciclo de vida de um projeto de software, contrastando metodologias tradicionais (Waterfall) e ágeis (Agile), e entenderemos como princípios de design (como modularidade, coesão e baixo acoplamento) impactam a legibilidade e a manutenibilidade do código ao longo do tempo. Prepare-se para pensar como um(a) engenheiro(a) ao abordar problemas de dados: após esta aula, você provavelmente enxergará seu código de um jeito bem diferente – e melhor!

E não tenha apego ao seu código, ele sempre pode ser melhorado e evoluído; portanto, deixe o orgulho de lado. Então, vamos em frente e não deixe de assistir às videoaulas para ver esses conceitos ganhando vida na prática.

HANDS ON

Nesta seção prática, você será apresentado(a) a um exemplo concreto de como as técnicas de engenharia de software podem melhorar um projeto de ciência de dados. Nestas videoaulas, vamos refatorar um pequeno código de análise de dados, inicialmente escrito de forma desorganizada, transformando-o em uma solução mais limpa e confiável.

A ideia é mostrar, passo a passo, como dividir um problema em funções reutilizáveis, lidar com exceções e validar comportamentos – tudo isso usando Python, uma das linguagens mais populares em ciência de dados. Fique atento(a) aos conceitos aplicados no vídeo: observar como um código “bagunçado” pode se tornar estruturado e fácil de manter é uma excelente forma de compreender na prática a importância de escrever um código de qualidade.

Para acompanhar, a seguir temos um trecho de código simplificado semelhante ao que é apresentado no hands on. Trata-se de uma função em Python que calcula a média de uma lista de valores, ignorando entradas inválidas (None). Esse exemplo ilustra alguns cuidados fundamentais: encapsular lógica em uma função (modularidade), documentar seu propósito e tratar casos excepcionais (neste caso, o que fazer se a lista filtrada ficar vazia).

Você pode executar este código e, em seguida, experimentar alterar alguns valores (inclusive inserindo ou removendo None) para ver como a função se comporta. Tente também criar casos de teste simples – por exemplo, verificar se `calcular_media([10, 20, 30])` retorna 20. Assim, você pratica não apenas escrever a função, mas também validar se ela funciona conforme o esperado. Vamos ao código:

```
# Linguagem: Python
def calcular_media(valores):
    """Calcula a média de uma lista de números, ignorando valores None."""
    filtrados = [x for x in valores if x is not None]
    if len(filtrados) == 0:
        return None # Evita divisão por zero se lista vazia após filtrar
    return sum(filtrados) / len(filtrados)

# Exemplo de uso da função:
dados = [10, None, 25, 40]
media = calcular_media(dados)
print(f"Média calculada: {media:.2f}" if media is not None else "Lista vazia após filtragem.")
```

Código-fonte 1 – Função Python para cálculo de média com tratamento de valores ausentes

Fonte: Elaborado pelo autor (2025)

Observe como `calcular_media` é pequena, focada e autocontida: ela não depende de variáveis globais e retorna um resultado claro a partir de entradas. No hands on, você verá como escrever testes unitários para funções assim e como integrá-las em um pequeno pipeline de análise.

Ao final do vídeo, desafie-se: pegue um código seu (por exemplo, um script de limpeza de dados) e tente aplicar ideias semelhantes – rode em seu ambiente, veja se ficou mais claro e confiável. Esse é o espírito: aprender fazendo. Assista às videoaulas antes de prosseguir, pois os próximos tópicos referem-se diretamente ao que foi demonstrado nesse exercício prático.

SAIBA MAIS

Engenharia de Software e Ciência de Dados: uma dupla inseparável. a ciência de dados acontece por meio de código – e, para que os resultados sejam confiáveis e reproduzíveis, o código precisa seguir padrões de qualidade. Muitas vezes, porém, cientistas de dados vêm de formações diversas e aprendem a programar focados(as) apenas em “fazer funcionar”. Quem nunca escreveu um código meio confuso, mas que gerava o gráfico desejado, e seguiu em frente? O problema é que, em projetos reais, “fazer funcionar uma vez” não basta. É preciso garantir que funcione amanhã, com dados novos, em ambiente diferente ou nas mãos de outro time.

A engenharia de software traz exatamente essa disciplina: é a área da computação que define métodos, ferramentas e boas práticas para o desenvolvimento sustentável de software, em contraste com a programação “ad hoc” ou improvisada. Uma frase famosa de um dos precursores da área, Ward Cunningham, compara atalhos no código a dívidas: cada gambiarra é como contrair uma dívida que, mais cedo ou mais tarde, cobrará juros – ou seja, exigirá retrabalho e correção onerosa no futuro.

Em 2015, engenheiros do Google publicaram um artigo intitulado “Machine Learning: The High-Interest Credit Card of Technical Debt” (O Cartão de Crédito de Juros Altos da Dívida Técnica) que alertava: sistemas de aprendizado de máquina tendem a acumular dívida técnica de forma ainda mais insidiosa que softwares tradicionais, pois incluem componentes extras como dados, modelos e pipelines complexos.

Em outras palavras, um projeto de ciência de dados improvisado pode rapidamente se tornar um emaranhado difícil de manter e evoluir. Por isso, começa a ficar claro nas empresas que bons modelos precisam vir acompanhados de bons códigos. Catherine Nelson, autora do livro “Software Engineering for Data Scientists”, afirma logo na introdução: “a habilidade de escrever código reproduzível, robusto e escalável é fundamental para o sucesso de um projeto de ciência de dados – absolutamente essencial para quem trabalha com código em produção”.

A boa notícia é que não se trata de “reinventar a roda”: as técnicas consagradas de engenharia de software (projeto modular, controle de versão, testes etc.) podem e devem ser aplicadas a projetos de Data Science, com as devidas adaptações. Cada vez mais, vemos a aproximação das duas áreas – surgem termos como MLOps (Machine Learning Operations), que nada mais é do que trazer práticas de DevOps (operações de desenvolvimento) para o ciclo de vida dos modelos de ML, incluindo implantação contínua, monitoramento e gestão de versões de modelos em produção.

Em resumo, se até recentemente um(a) cientista de dados podia até se orgulhar de “não seguir padrões de engenheiro de software” e mesmo assim entregar análises, hoje isso mudou: projetos de dados de sucesso exigem um mínimo de excelência em engenharia, seja você ou não formalmente um/uma engenheiro(a) de software. E se isso parece muita coisa pra se preocupar além dos modelos, lembre-se que código de qualidade economiza tempo a médio e longo prazo – menos bugs para resolver, menos dores de cabeça ao escalar, menos falhas catastróficas no meio de uma entrega importante.

Ciclo de vida de desenvolvimento: do Waterfall ao Agile (e ao CRISP-DM). Todo software – inclusive aquele script Python que gera um insight de dados – passa por etapas em seu desenvolvimento. De modo geral, podemos delinear um ciclo de vida clássico:

- 1) Concepção (entendimento do problema e requisitos).
- 2) Desenvolvimento (codificação da solução).
- 3) Testes (verificação e validação se o software funciona como esperado).
- 4) Implantação (colocar em uso, seja em um produto interno, em um aplicativo web etc.).
- 5) Manutenção (corrigir erros descobertos, adaptar a novas necessidades, melhorar desempenho etc.).

Modelos tradicionais como o Waterfall (cascata) sugerem executar essas fases em sequência linear: primeiro coleta-se todos os requisitos, depois parte-se para implementar tudo, depois testa-se tudo e assim por diante. Esse modelo,

embora organizado, tem pouca flexibilidade: qualquer mudança de escopo no meio do caminho “quebra” o fluxo, já que não há retorno previsto para fases anteriores.

Em projetos de ciência de dados, essa rigidez é particularmente problemática – é comum, por exemplo, descobrir durante a fase de modelagem que os dados disponíveis não atendem ao objetivo, o que obriga a voltar à etapa de obtenção/entendimento de dados. Por isso, metodologias ágeis ganharam espaço. No Agile, as etapas ainda existem, mas são percorridas em ciclos curtos e iterativos, chamados sprints. Em vez de um plano fechado de 6 meses, planeja-se entregas incrementais a cada 1 ou 2 semanas, por exemplo, permitindo incorporar feedback contínuo.

Pense assim: em ciência de dados, muitas vezes não sabemos de antemão qual modelo dará certo; com uma abordagem ágil, podemos criar uma solução simples inicial (um baseline), mostrar ao cliente ou usuário para obter feedback rápido e então aprimorar gradualmente. Essa interação constante com stakeholders diminui o risco de passar meses desenvolvendo algo que não atende à necessidade real.

No contexto de projetos de Data Science, existe inclusive um ciclo de vida específico muito difundido, o CRISP-DM (Cross-Industry Standard Process for Data Mining). Ele define fases como Entendimento do Negócio, Entendimento dos Dados, Preparação dos Dados, Modelagem, Avaliação e Deploy (implantação) – dispostas de forma cíclica, indicando que o processo é iterativo e pode voltar etapas conforme necessário. A figura a seguir ilustra bem como o ciclo funciona.

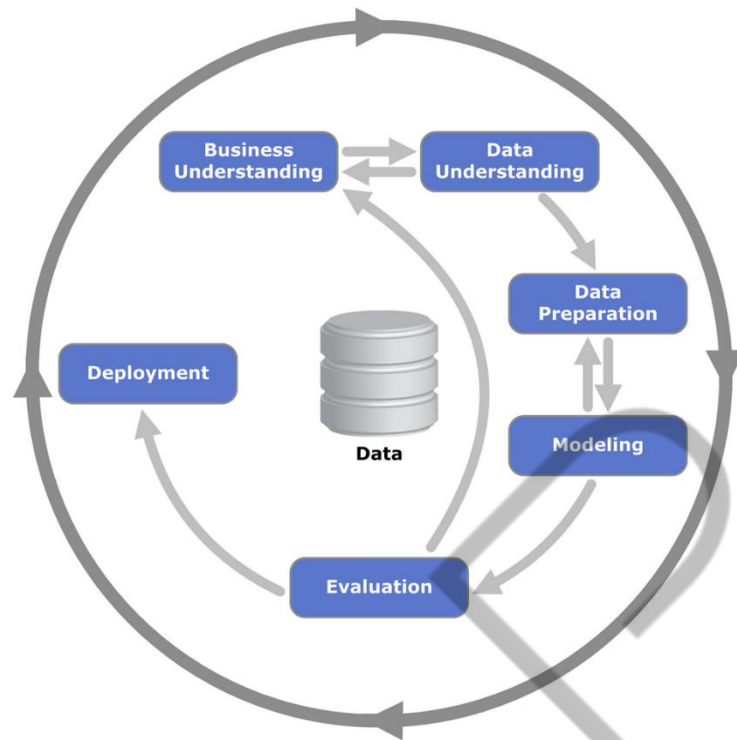


Figura 1 – CRISP-DM Process Diagram
Fonte: Wikipedia Commons (2012)

Por exemplo, durante a Avaliação (fase em que você verifica se o modelo atinge os objetivos de negócio), pode-se concluir que é preciso coletar mais dados ou tentar uma abordagem de modelagem diferente, retornando assim às etapas anteriores. O CRISP-DM foi concebido lá no fim dos anos 1990, mas continua relevante e está alinhado com a mentalidade ágil: é um ciclo adaptativo e centrado em aprendizado e melhorias contínuas.

Grandes empresas adaptaram esses conceitos: a Microsoft, por exemplo, propôs o TDSP (Team Data Science Process), que adiciona práticas de engenharia e DevOps ao CRISP-DM, enfatizando implantação contínua e monitoramento de modelos. Tudo isso converge para a noção atual de MLOps: integrar o processo de desenvolvimento do modelo ao de software, com pipelines automatizados de treinamento e deployment, ambientes padronizados etc.

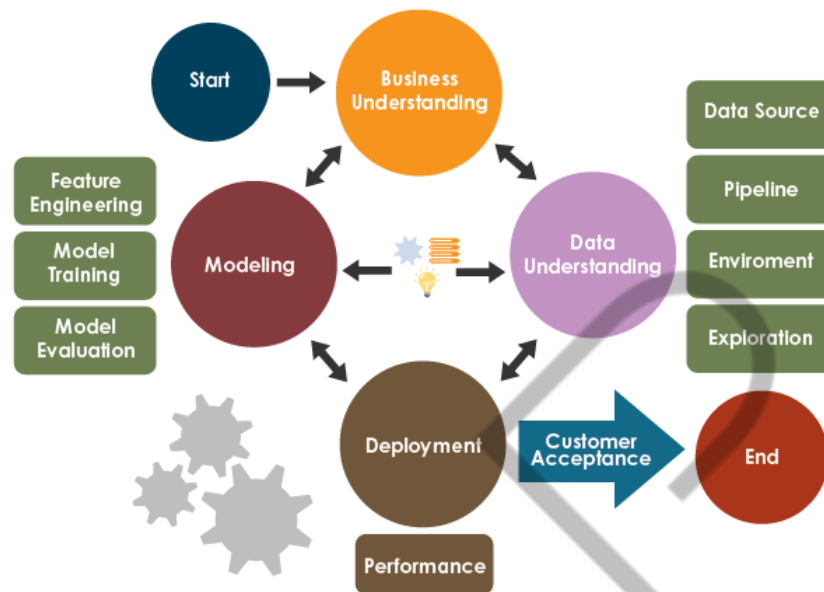


Figura 2 – Ciclo TDSP
Fonte: Life Cycle (2025)

Uma analogia útil é pensar que projetos de ciência de dados combinam elementos de pesquisa experimental e de desenvolvimento de produto de software. Como pesquisa, requer experimentação e evolução (logo, beneficiam-se de agilidade e ciclos iterativos); como produto de software, requer confiabilidade, manutenção e colaboração em equipe (logo, beneficiam-se de planejamento e boas práticas de engenharia). Não é por acaso que muitas equipes de dados hoje participam de cerimônias ágeis – dailys, sprint reviews, retrospectivas –, adaptando-as à sua realidade.

Por exemplo, a definição de “pronto” para uma tarefa de modelagem pode ser algo como “modelo com acurácia X validada, código revisado e com testes unitários dos componentes críticos”; se a meta não for atingida, registra-se como aprendizado e planeja-se a próxima iteração com novos experimentos. A seguir, resumimos algumas diferenças entre os modelos Waterfall e Agile e sua relevância para a ciência de dados.

ETAPA	WATERFALL (CASCATA)	AGILE (ÁGIL)
Planejamento	Extenso, inicial e fixo. Cada fase segue um plano pré-definido; mudança de escopo é indesejada.	Incremental e contínuo. Requisitos podem evoluir a cada sprint; aceita-se mudanças conforme feedback e descobertas.
Iterações	Sem iterações formais – projeto visto como um bloco único; iteração só ocorreria iniciando um novo projeto.	Altamente iterativo – trabalho dividido em sprints curtos. Permite revisitar e aprimorar funcionalidades diversas vezes.
Adaptabilidade	Baixa. Se dados ou objetivos mudam durante o projeto, é difícil se adaptar sem atrasos significativos.	Alta. O processo espera mudanças: se um modelo não funciona bem, o time ajusta prioridades na próxima sprint (ex.: coleta mais dados, tenta algoritmo diferente).
Aplicação em Ciência de Dados	Pouco compatível. Fases rígidas não lidam bem com natureza exploratória da ciência de dados.	Muito compatível. Iteração permite experimentação e refinamento contínuo de modelos e análises, integrando feedback de negócio rapidamente.

Tabela 1 – Comparação entre abordagens Waterfall e Ágil no desenvolvimento de projetos (ênfase em implicações para ciência de dados)
Fonte: Elaborado pelo autor (2025)

Como indica a tabela, métodos ágeis oferecem a flexibilidade necessária para projetos de ciência de dados sem abrir mão da organização. Mas isso não significa caos: agile não é “cada um faz o que quer sem documentar nada” – ao contrário, exige muita comunicação e disciplina (escrevendo user stories, critérios de aceitação, mantendo backlog atualizado etc.).

O ponto central é: projetos de ciência de dados se beneficiam de um processo iterativo com retroalimentação constante. Portanto, sempre que iniciar uma nova empreitada de ciência de dados, pense em como dividir o trabalho em incrementos entregáveis (por exemplo, primeiro entregar um relatório exploratório simples, depois uma prova de conceito de modelo, depois uma versão refinada com validação cruzada e assim por diante).

Chamando para a ação: se você tiver interesse em ver exemplos reais, recomendo buscar a metodologia do CRISP-DM – muitos recursos online (artigos e vídeos) explicam como aplicar esse ciclo iterativo na prática. E claro, lembre-se de acompanhar o planejamento ágil do seu próprio projeto de pós-graduação: como você está organizando suas semanas de trabalho? Experimente adotar mini-sprints de estudo e desenvolvimento, com objetivos claros, e veja a diferença.

Princípios fundamentais de design de código: modularidade, coesão e baixo acoplamento. Agora entramos no “coração” técnico da engenharia de software: como estruturar o código internamente para que ele seja fácil de entender, manter e evoluir. Dois conceitos-chave aqui são coesão e acoplamento. Vamos definir de forma simples:

Modularidade: é a ideia de dividir o software em componentes ou módulos menores, cada um responsável por uma parte da funcionalidade total. Em termos práticos, significa escrever funções, classes e módulos (arquivos) que tenham objetivos claros. Por exemplo, em um projeto de data science, você pode ter um módulo só para tratamento e limpeza de dados, outro para treinamento de modelos, outro para avaliar desempenho etc.

A modularidade segue o famoso mantra “dividir para conquistar”: um código em módulos menores é mais fácil de testar isoladamente, reutilizar em outros projetos e modificar sem arruinar todo o sistema. Além disso, permite que times diferentes trabalhem em módulos distintos simultaneamente, facilitando a colaboração. Agora veja um exemplo ruim e um bom.

Exemplo Ruim (Monolítico - Sem Modularidade):

```
# Linguagem: Python
# Arquivo: processar_tudo.py
# (Baixa modularidade)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Carregar dados (Responsabilidade 1)
print("Carregando dados...")
try:
    dados = pd.read_csv('meus_dados.csv')
except FileNotFoundError:
    print("Arquivo não encontrado!")
    exit()

# 2. Limpar dados (Responsabilidade 2)
print("Limpendo dados...")
dados = dados.dropna() # Remove linhas com NaN
dados['nova_coluna'] = dados['coluna_existente'] * 2 # Feature engineering

# 3. Treinar modelo (Responsabilidade 3)
print("Treinando modelo...")
X = dados.drop('target', axis=1)
y = dados['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

# 4. Avaliar modelo (Responsabilidade 4)
print("Avaliando modelo...")
preds = modelo.predict(X_test)
precisao = accuracy_score(y_test, preds)

print(f"Precisão do modelo: {precisao}")
```

Código-fonte 2 – Exemplo de Modularidade em Python
Fonte: Elaborado pelo autor (2025)

Notou o problema? É difícil reutilizar só a parte de "Limpar dados" em outro projeto. Se o LogisticRegression mudar, você mexe no mesmo arquivo que carrega o CSV.

Exemplo Bom (Modularizado):

Agora, dividimos as responsabilidades em arquivos diferentes.

```
# Linguagem: Python
# Arquivo: utils_dados.py
# (Módulo 1: Focado em dados)

import pandas as pd

def carregar_dados(caminho):
    """Carrega dados de um CSV."""
    try:
        return pd.read_csv(caminho)
    except FileNotFoundError:
        print(f"Erro: Arquivo {caminho} não encontrado.")
        return None

def limpar_dados(df):
    """Aplica limpeza e feature engineering."""
    df_limpo = df.dropna()
    df_limpo['nova_coluna'] = df_limpo['coluna_existente'] * 2
    return df_limpo
```

Código-fonte 3 – Exemplo de Modularidade em Python
Fonte: Elaborado pelo autor (2025)

```
# Linguagem: Python
# Arquivo: utils_modelo.py
# (Módulo 2: Focado em ML)

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def treinar_modelo(X, y):
    """Treina um modelo de regressão logística."""
    modelo = LogisticRegression()
    modelo.fit(X, y)
    return modelo

def avaliar_modelo(modelo, X_test, y_test):
    """Calcula a precisão do modelo."""
    preds = modelo.predict(X_test)
    return accuracy_score(y_test, preds)
```

Código-fonte 4 – Exemplo de Modularidade em Python
Fonte: Elaborado pelo autor (2025)

```
# Linguagem: Python
# Arquivo: main.py
# (Módulo 3: O "orquestrador" que usa os outros)

# Importa as funções dos nossos módulos
import utils_dados
import utils_modelo

print("Iniciando pipeline...")

# 1. Dados
dados_brutos = utils_dados.carregar_dados('meus_dados.csv')
if dados_brutos is not None:
    dados_limpos = utils_dados.limpar_dados(dados_brutos)

# 2. Modelo
X = dados_limpos.drop('target', axis=1)
y = dados_limpos['target']
# (Dividir os dados também poderia estar em utils_modelo)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

modelo = utils_modelo.treinar_modelo(X_train, y_train)

# 3. Avaliação
precisao = utils_modelo.avaliar_modelo(modelo, X_test, y_test)
print(f"Precisão final: {precisao}")
```

Código-fonte 5 – Exemplo de Modularidade em Python
Fonte: Elaborado pelo autor (2025)

Notou a diferença? Se você quiser trocar a LogisticRegression por uma RandomForest, você só mexe no utils_modelo.py. O main.py nem fica sabendo da mudança. Vai ser menos problema para ser resolvido por seu “eu” do futuro.

Coesão: mede o quão bem as partes internas de um módulo estão relacionadas ao mesmo propósito. Dizemos que um módulo tem alta coesão quando tudo que ele faz converge para uma única tarefa ou conceito.

Por exemplo, imagine uma função que deve calcular estatísticas descritivas (média, mediana, variância) de um conjunto de dados. Se ela se limitar a isso e retornar esses números, ótimo – alta coesão. Mas se dentro dessa função alguém aproveita, também gera um gráfico e salva em disco, além de enviar um e-mail com os resultados, então ela está acumulando responsabilidades diferentes (cálculo, visualização, I/O de arquivo, notificação) – ou seja, baixa coesão.

Veja os exemplos a seguir:

```
# (Baixa coesão)
import matplotlib.pyplot as plt
import smtplib # Biblioteca de email

def calcular_estatisticas_e_notificar(lista_numeros):
    """
    Esta função tem baixa coesão.
    Ela mistura 4 responsabilidades:
    1. Cálculo (estatística)
    2. Geração de gráfico (visualização)
    3. Salvar arquivo (I/O)
    4. Enviar e-mail (notificação)
    """

    # 1. Cálculo
    media = sum(lista_numeros) / len(lista_numeros)
    mediana = sorted(lista_numeros)[len(lista_numeros) // 2]
    print(f"Média: {media}, Mediana: {mediana}")

    # 2. Geração de gráfico
    plt.hist(lista_numeros)
    plt.title("Histograma dos Dados")

    # 3. Salvar arquivo
    caminho_grafico = 'grafico_hoje.png'
    plt.savefig(caminho_grafico)
    print(f"Gráfico salvo em {caminho_grafico}")

    # 4. Enviar e-mail
    servidor = smtplib.SMTP('smtp.exemplo.com')
    servidor.sendmail('remetente@exemplo.com',
                     'destinatario@exemplo.com',
                     f'Assunto: Relatório\n\nResultados: Média={media}')
    servidor.quit()
    print("E-mail enviado.")

# Uso:
dados = [1, 2, 2, 3, 3, 3, 4, 4, 5, 10]
calcular_estatisticas_e_notificar(dados)
```

Código-fonte 6 – Exemplo de Coesão em Python
Fonte: Elaborado pelo autor (2025)

```
# (Alta coesão)
import matplotlib.pyplot as plt
import smtplib

# Função 1: Focada 100% em cálculo
def calcular_estatisticas(lista_numeros):
    """Retorna um dicionário com estatísticas descritivas."""
    media = sum(lista_numeros) / len(lista_numeros)
    mediana = sorted(lista_numeros)[len(lista_numeros) // 2]
    return {"media": media, "mediana": mediana}

# Função 2: Focada 100% em visualização
def gerar_histograma(lista_numeros, caminho_saida):
    """Gera e salva um histograma dos dados."""
    plt.hist(lista_numeros)
    plt.title("Histograma dos Dados")
    plt.savefig(caminho_saida)
    print(f"Gráfico salvo em {caminho_saida}")

# Função 3: Focada 100% em notificação
def enviar_email_relatorio(stats_dict):
    """Envia um e-mail com os resultados."""
    corpo_email = f'Assunto: Relatório\n\nResultados: Média={stats_dict["media"]}'

    servidor = smtplib.SMTP('smtp.exemplo.com')
    servidor.sendmail('remetente@exemplo.com',
                     'destinatario@exemplo.com',
                     corpo_email)
    servidor.quit()
    print("E-mail enviado.")

# Uso (Agora o "orquestrador" decide o que fazer):
dados = [1, 2, 2, 3, 3, 3, 4, 4, 5, 10]

# 1. Podemos só calcular
estatisticas = calcular_estatisticas(dados)
print(f"Estatísticas: {estatisticas}")

# 2. Podemos calcular E gerar gráfico
gerar_histograma(dados, 'grafico_final.png')

# 3. Podemos calcular E enviar e-mail
enviar_email_relatorio(estatisticas)
```

Código-fonte 7 – Exemplo de Coesão em Python
Fonte: Elaborado pelo autor (2025)

Olha que interessante: agora você pode reutilizar `calcular_estatisticas` em qualquer lugar, sem o "peso" de ter que gerar um gráfico ou enviar um e-mail.

Boas práticas pedem alta coesão, pois quando cada parte tem uma responsabilidade clara, o código fica mais legível (sabemos exatamente o que esperar de cada função) e mantível (alterações pontuais têm impacto limitado). A ideia da *single responsibility* (responsabilidade única) de cada módulo é fundamental: cada função ou classe deve ser “especialista” em algo, não um faz-tudo.

Acoplamento: refere-se ao grau de interdependência entre módulos. Se alterações no módulo A frequentemente exigem mudanças no módulo B, ou se A

não funciona sem B sempre rodando junto, dizemos que há alto acoplamento entre A e B. Baixo acoplamento, por sua vez, é quando os módulos interagem por interfaces bem definidas e minimamente necessárias.

Por exemplo, suponha que você tem uma função que limpa dados e outra que treina um modelo. Se a função de treinamento “puxa” dados diretamente de variáveis globais definidas na função de limpeza, há um acoplamento forte e implícito entre elas (ordem de execução fixa, dependência de estado global etc.). Melhor seria a função de limpeza retornar os dados limpos e a de treinamento aceitar os dados como parâmetro – assim, elas se comunicam apenas via os dados necessários, podendo existir mais independentemente.

Em design de código, almejamos acoplamento fraco: módulos conversam apenas o necessário e preferencialmente por meio de interfaces estáveis (como parâmetros de função, objetos de transferência, APIs). Com baixo acoplamento, se amanhã decidirmos trocar a estratégia de limpeza de dados (digamos, usar uma biblioteca nova), podemos fazê-lo sem ter que reescrever a rotina de treinamento – bastando garantir que a nova limpeza ainda fornece os dados no formato esperado.

```
# (Alto Acoplamento)
class ServicoDeEmail: """ Uma classe CONCRETA que sabe enviar e-mails usando uma biblioteca
específica (ex: smtplib). """ def enviar_email(self, destinatario, assunto, corpo):
print(f"Conectando ao servidor SMTP...") print(f"Enviando e-mail para {destinatario}: {assunto}")
print(f"Corpo: {corpo}") print("E-mail enviado com sucesso.") return True
class ProcessadorDePedidos: """ Esta classe está FORTEMENTE ACOPLADA ao ServicoDeEmail. """ def
processar_pedido(self, pedido): print(f"Processando pedido {pedido['id']}...") # ...lógica de
negócio...
# O "PECADO": O processador instancia concretamente o serviço.
# Ele "sabe" como construir um ServicoDeEmail.
notificador = ServicoDeEmail()

try:
    notificador.enviar_email(
        pedido['cliente_email'],
        "Seu pedido foi confirmado!",
        f"Detalhes do pedido: {pedido['id']}"
    )
    print(f"Pedido {pedido['id']} finalizado.")
except Exception as e:
    print(f"Falha ao notificar: {e}")
----- Uso -----
meu_pedido = {'id': '123A', 'cliente_email': 'diogo@email.com'} processador =
ProcessadorDePedidos() processador.processar_pedido(meu_pedido)
```

Código-fonte 8 – Exemplo de Acoplamento em Python
Fonte: Elaborado pelo autor (2025)

Por que isso é ruim?

- É impossível de Mudar! E se amanhã a empresa decidir que é melhor enviar um SMS ou uma mensagem no Slack em vez de e-mail? Você terá que abrir o código do `ProcessadorDePedidos` e alterar a linha `notificador = ServicoDeEmail()` para `notificador = ServicoDeSMS()`, adicionando um `if/else`. Isso quebra o princípio "Aberto/Fechado".
- É impossível de Testar! Como você pode rodar um teste unitário no `ProcessadorDePedidos` sem que ele tente enviar um e-mail de verdade e se conectar a um servidor SMTP? É muito difícil. O seu teste de "lógica de pedido" agora está acoplado a um "serviço de rede".

E como esse problema pode ser resolvido?

O problema é resolvido aplicando o Princípio da Inversão de Dependência (o "D" de SOLID).

Em vez de o `ProcessadorDePedidos` (módulo de alto nível) depender diretamente da classe concreta `ServicoDeEmail` (módulo de baixo nível), ambos passarão a depender de uma abstração (uma "interface" ou contrato).

A solução prática é a Injeção de Dependência: o `ProcessadorDePedidos` não vai mais criar o serviço de notificação; ele vai receber esse serviço pronto em seu construtor (`__init__`).

Veja o código refatorado para ter Baixo Acoplamento:

1. O Contrato (Abstração)

Primeiro, definimos um "contrato" que qualquer serviço de notificação deve seguir. Em Python, usamos uma Classe Base Abstrata (ABC).

```
from abc import ABC, abstractmethod

class IServicoNotificacao(ABC):
    """
    Esta é a ABSTRAÇÃO (ou Interface).
    Ela define o "o quê" (o contrato), mas não o "como".
    Qualquer classe que quiser ser um notificador DEVE ter este método.
    """
    @abstractmethod
    def enviar_notificacao(self, destinatario, assunto, corpo):
        pass
```

Código-fonte 9 – Exemplo de Acoplamento em Python
Fonte: Elaborado pelo autor (2025)

2. Implementações Concretas

Agora, criamos nossas classes "reais" que implementam esse contrato.

```
class ServicoDeEmail(IServicoNotificacao):
    """
    Uma implementação CONCRETA do contrato.
    Esta classe sabe enviar e-mails.
    """
    def enviar_notificacao(self, destinatario, assunto, corpo):
        print("--- [E-MAIL] ---")
        print(f"Conectando ao servidor SMTP...")
        print(f"Enviando para: {destinatario}")
        print(f"Assunto: {assunto}")
        print(f"Corpo: {corpo}")
        print("-----")
        return True

class ServicoDeSMS(IServicoNotificacao):
    """
    OUTRA implementação CONCRETA.
    Esta classe sabe enviar SMS.
    """
    def enviar_notificacao(self, destinatario, assunto, corpo):
        # O 'assunto' pode ser ignorado ou prefixado no corpo do SMS
        print("--- [SMS] ---")
        print(f"Conectando ao Gateway de SMS...")
        print(f"Enviando para: {destinatario} (telefone)")
        print(f"Mensagem: {assunto} - {corpo}")
        print("-----")
        return True
```

Código-fonte 10 – Exemplo de Acoplamento em Python
Fonte: Elaborado pelo autor (2025)

3. A Classe Principal (Desacoplada)

Aqui está a mágica. O `ProcessadorDePedidos` agora recebe a dependência no construtor e depende apenas da abstração (`IServicoNotificacao`).

```
class ProcessadorDePedidos:
    """
    Esta classe agora está FRACAMENTE ACOPLADA.
    Ela não sabe se vai enviar E-mail, SMS ou Slack.
    Ela só sabe que vai usar "algo" que obedece ao contrato IServicoNotificacao.
    """

    # 1. A dependência é "injetada" aqui (Injeção de Dependência)
    def __init__(self, servico_notificacao: IServicoNotificacao):
        self._notificador = servico_notificacao

    def processar_pedido(self, pedido):
        print(f"Processando pedido {pedido['id']}...")
        # ...lógica de negócio...

        # 2. Não há mais "ServicoDeEmail()"!
        # Ele usa o serviço que foi injetado, seja ele qual for.
        try:
            self._notificador.enviar_notificacao(
                pedido['cliente_contato'],
                "Seu pedido foi confirmado!",
                f"Detalhes do pedido: {pedido['id']}"
            )
        )
```

```
print(f"Pedido {pedido['id']} finalizado.")
except Exception as e:
    print(f"Falha ao notificar: {e}")
```

Código-fonte 11 – Exemplo de Acoplamento em Python

Fonte: Elaborado pelo autor (2025)

4. Juntando Tudo (Injeção)

Quem decide qual serviço usar é o código "externo" que monta a aplicação.

```
# Dados do pedido (note que o contato pode ser e-mail ou telefone)
meu_pedido_email = {'id': '123A', 'cliente_contato': 'diogo@email.com'}
meu_pedido_sms = {'id': '456B', 'cliente_contato': '+5511999998888'}

# ----- CENÁRIO 1: A empresa usa E-mail -----
print("### EXECUTANDO COM E-MAIL ###")
# 1. Criamos a implementação concreta de E-mail
notificador_email = ServicoDeEmail()
# 2. Injetamos ela no processador
processador_com_email = ProcessadorDePedidos(servico_notificacao=notificador_email)
# 3. Executamos
processador_com_email.processar_pedido(meu_pedido_email)

print("\n" + "="*30 + "\n")

# ----- CENÁRIO 2: A empresa mudou para SMS -----
print("### EXECUTANDO COM SMS ###")
# 1. Criamos a implementação concreta de SMS
notificador_sms = ServicoDeSMS()
# 2. Injetamos ela no processador
processador_com_sms = ProcessadorDePedidos(servico_notificacao=notificador_sms)
# 3. Executamos (sem NENHUMA mudança no ProcessadorDePedidos!)
processador_com_sms.processar_pedido(meu_pedido_sms)
```

Código-fonte 12 – Exemplo de Acoplamento em Python

Fonte: Elaborado pelo autor (2025)

Note que é fácil de mudar! Se amanhã você quiser enviar pelo Slack, basta criar a classe `ServicoDeSlack(IServicoNotificacao)` e injetá-la no `ProcessadorDePedidos`. Você não precisa modificar o código do `ProcessadorDePedidos`. Isso respeita o Princípio Aberto/Fechado.

É fácil de testar! Para testar o `ProcessadorDePedidos` sem enviar um e-mail de verdade, você pode criar um "Mock" (um dublê de teste) e injetá-lo.

Exemplo de Teste Unitário:

```
class ServicoDeNotificacaoMock(IServicoNotificacao):
    """
    Um notificador FALSO, usado APENAS para testes.
    Ele não envia nada, apenas registra que foi chamado.
    """
```

```
"""
def __init__(self):
    self.notificacoes_enviadas = []

def enviar_notificacao(self, destinatario, assunto, corpo):
    print("--- [MOCK] ---")
    print("Notificação Falsa capturada (NADA FOI ENVIADO).")
    print("-----")
    self.notificacoes_enviadas.append({
        'destinatario': destinatario,
        'assunto': assunto,
        'corpo': corpo
    })
    return True

# ----- Rodando o Teste -----
print("\n" + "="*30 + "\n")
print("### EXECUTANDO TESTE UNITÁRIO ###")

# 1. Criamos o Mock
notificador_falso = ServicoDeNotificacaoMock()
# 2. Injetamos o Mock no processador
processador_para_teste = ProcessadorDePedidos(servico_notificacao=notificador_falso)

# 3. Executamos a lógica de negócio
pedido_teste = {'id': 'TESTE999', 'cliente_contato': 'teste@unitario.com'}
processador_para_teste.processar_pedido(pedido_teste)

# 4. Verificamos se o processador fez o trabalho dele
# (chamar o notificador corretamente) sem depender da rede.
assert len(notificador_falso.notificacoes_enviadas) == 1
assert notificador_falso.notificacoes_enviadas[0]['assunto'] == "Seu pedido foi confirmado!"

print("\nTeste passou! O processador foi testado em isolamento.")
```

Código-fonte 13 – Exemplo de Acoplamento em Python
Fonte: Elaborado pelo autor (2025)

Com essa abordagem, seu `ProcessadorDePedidos` agora é flexível, testável e não tem mais ideia de como as notificações são realmente enviadas, o que é a definição de baixo acoplamento.

Resumindo: módulos com alta coesão e baixo acoplamento. Isso resulta em sistemas em que as peças são independentes e plugáveis. Na prática da ciência de dados, como aplicamos isso? Pense em pipelines de processamento: é comum termos etapas como extração dos dados, transformação, treinamento, validação, relatório. Cada uma dessas etapas pode ser implementada como função ou classe separada, com interfaces bem definidas (ex.: função `transformar_dados(dados_brutos) -> retorna dados_transformados`).

Se cada etapa é tratada como módulo, você pode, por exemplo, atualizar a função de transformação (talvez adicionando normalização ou codificação diferente) sem precisar tocar nas outras funções, a não ser que a interface (o que ela recebe e

o que devolve) mude. Essa separação também facilita o reuso: se em um novo projeto você precisa de algo parecido, pode pegar só o módulo de interesse.

Um benefício colateral importante é no debug e testes: com módulos independentes, fica muito mais simples isolar problemas. Se um relatório final sai incorreto, você pode testar cada etapa separadamente (garantir que os dados foram lidos corretamente, depois que as transformações funcionaram, etc.) e achar exatamente onde o erro foi introduzido. Em um código monolítico, onde tudo está junto, esses rastreamentos são muito mais desafiadores.

Para enfatizar, imagine um caso real: um dos motivos pelos quais muitos projetos de Machine Learning não escalam é o excesso de “glue code” – código “cola”, escrito para ligar um componente ao outro sem seguir um design claro. Por exemplo, scripts que carregam um modelo aqui, transformam um tipo de dado ali, chamam um serviço acolá, tudo misturado.

Um estudo do Google mostrou que em sistemas de ML maduros, o código de aprendizado em si (o modelo) costuma ser menos de 5% do total, enquanto 95% é infraestrutura e código auxiliar (ETL, validação, monitoramento, interfaces). Se esse código auxiliar for mal estruturado, a dívida técnica do sistema cresce exponencialmente, dificultando qualquer melhoria. A solução? Arquiteturas modulares, componentes fracamente acoplados e bem integrados – somente assim aquele 5% (o modelo) poderá ser atualizado ou substituído sem quebrar o resto.

Para fechar este tópico: quando for escrever suas próximas análises, tente mapear quais são as partes lógicas do problema (coleta de dados, limpeza, análise exploratória, modelagem, visualização...) e certifique-se de que seu código reflete essa divisão. Se você usar Notebooks, por exemplo, separe por seções bem definidas e evite dependência excessiva de uma célula em estado mutável de outra – notebooks podem incentivar acoplamento temporal (ordem fixa de execução), mas você pode impor ordem lógica ordenando as seções e limpando o estado. Em código Python puro, aproveite para criar funções.

Experimente: pegue um script longo que você tenha e refatore criando funções coesas. Execute e veja se tudo continua funcionando. Depois, mostre para um colega: é bem possível que agora ele/ela consiga entender melhor seu fluxo de trabalho comparado à versão original.

Boas práticas essenciais: versionamento, documentação, testes e revisão – qualidade e manutenção de código. Além do design estrutural, a engenharia de software preconiza um conjunto de práticas no dia a dia do desenvolvimento que garantem qualidade e facilitam a colaboração. Vamos passar pelas principais e contextualizar no ambiente de ciência de dados.

Controle de versão (Git e afins): é difícil exagerar a importância de usar controle de versão em projetos de código, incluindo projetos de data science. Ferramentas como Git permitem acompanhar historicamente as mudanças no código, trabalhar em paralelo com outros(as) desenvolvedores(as) e integrar contribuições de forma controlada. Se você já se deparou com arquivos tipo `analise_final.py`, `analise_final_v2.py`, `analise_final_bkp_junho.py`, sabe o caos que é gerenciar versões manualmente.

Com um repositório Git, cada alteração significativa é commitada com mensagem de log; você pode rever evoluções, comparar diferenças, reverter caso algo dê errado e criar ramificações (branches) para desenvolver novas features ou experimentos sem arriscar estragar o código principal (branch main ou master). Em ciência de dados, isso significa, por exemplo, poder desenvolver um novo modelo em uma branch separada, enquanto outra pessoa ajusta o pipeline de preprocessing em outra branch – depois, com uma pull request, integrar cuidadosamente as duas evoluções, revisando se estão compatíveis.

Versionamento também abrange dados e modelos: ferramentas de Data Version Control (como DVC), ou até mesmo organização manual de datasets com versionamento (ex.: naming com datas), ajudam a reproduzir resultados. Lembre-se: um dos pilares da ciência é a reprodutibilidade, e sem controle de versão do código fica quase impossível saber qual versão do código gerou determinado resultado. Portanto, incorpore Git no seu fluxo se ainda não o fez – a curva de aprendizado compensa. Muitas plataformas (GitHub, GitLab, Bitbucket) oferecem integração contínua (CI) que pode rodar testes automáticos a cada mudança no repositório, garantindo que nada essencial quebrou – mais um ganho de qualidade.

Documentação e legibilidade: código bom se documenta em dois níveis. Primeiro, pelo próprio código autoexplicativo: escolha de nomes claros para variáveis e funções, organização de arquivos, formatação consistente. Seguir um guia de estilo (como o PEP8 para Python) ajuda todos a lerem o código sem

estranheza. Segundo, pela documentação explícita: docstrings em funções explicando o que fazem, comentários esclarecendo trechos complexos (mas evite comentar o óbvio) e documentos externos quando necessário (um README.md no repositório explicando como executar o projeto, por exemplo).

Em ciência de dados, um aspecto importante da documentação é registrar hipóteses e decisões: por exemplo, anotar porque você removeu certos outliers ou escolheu determinado modelo. Pode ser em comentários no notebook ou, melhor ainda, em um documento de suporte. Isso evita a famosa situação “por que mesmo eu fiz assim?” semanas depois. Em contextos colaborativos, a documentação é um salva-vidas para quem herdar seu código ou se juntar ao projeto no meio.

Pense também em notebooks bem narrados: usar células de texto para contextualizar as células de código torna suas análises mais compreensíveis (é como escrever um relatório dentro do notebook). Uma dica valiosa: pratique Clean Code – existe uma literatura vasta sobre escrever código limpo e legível (Robert C. Martin, “Clean Code”, é uma referência clássica).

Em resumo: prefira clareza à brevidade engenhosa. Um código Python ultra “pythonico” em uma linha que só você entende não é vantagem se outros não conseguirem dar manutenção; mais vale algumas linhas bem escritas com nomes elucidativos. E não esqueça de atualizar a documentação quando o código mudar, a fim de evitar discrepâncias.

Testes (automáticos de preferência): em engenharia de software, testes de unidade, integração e sistema fazem parte do desenvolvimento. No universo de data science, ainda encontramos alguma resistência de profissionais em escrever testes automatizados – muitas vezes porque o fluxo é exploratório e não evidente de testar. Mas é perfeitamente possível (e recomendável) testar partes do seu código. Escreveu uma função que calcula uma métrica? Faça um teste unitário conferindo se, para uma entrada conhecida, a saída está correta.

Por exemplo, a função `calcular_media` do hands on poderia ter um teste verificando que `calcular_media([1,2,3,None]) == 2` e que `calcular_media([None, None]) == None`. Testes atuam como um seguro: se alguém “otimizar” a função no futuro e introduzir sem querer um bug (talvez esquecendo de ignorar None), o teste vai falhar, alertando sobre a regressão. Em pipelines de dados, você pode testar, por

exemplo, se uma função de limpeza realmente remove ou traduz os valores esperados (dado um dataset fictício de entrada e o esperado de saída).

Testes de integração são um pouco mais complexos – por exemplo, simular o pipeline inteiro com um subconjunto pequeno de dados, ver se roda até o fim sem erros e produz um resultado consistente (pode ser um arquivo de saída com formato correto). Com testes no lugar, você ganha confiança para refatorar código: sabe rapidamente se algo importante quebrou. Ferramentas como unittest ou pytest no Python facilitam escrever e organizar testes.

Além disso, existe o conceito de testes de regressão para ciência de dados: salvar resultados de referência (por exemplo, métricas de um modelo atual) e ter testes que alertem se um novo modelo piorou drasticamente alguma métrica – assim você evita deployar sem perceber um modelo com desempenho inferior. E claro, testes manuais e exploratórios continuam sendo importantes: inspeção visual de gráficos, verificação estatística de outputs etc. Mas automatizar o que for possível é um grande passo rumo à confiabilidade.

Revisão de código (code review): “quatro olhos veem mais que dois”. Ter um(a) colega revisando seu código antes de integrá-lo ao repositório principal é uma prática extremamente valiosa. O processo de code review consiste em alguém ler seu código, entender o que ele faz e sugerir melhorias ou apontar problemas. Em uma equipe de data science, isso propicia compartilhamento de conhecimento (o revisor aprende sobre aquela parte do projeto) e padronização (seu código passa pelo crivo de alguém que pode pedir para seguir certo padrão que o time adota).

Além disso, reviewers muitas vezes detectam bugs ou casos não tratados que escaparam do autor original. Em termos de aprendizado, submeter seu código à revisão força você a escrever de forma mais clara (afinal, sabe que outra pessoa vai ler). Algumas organizações de pesquisa acadêmica têm adotado code review como forma de garantir qualidade de análises – algo relativamente novo no meio científico, mas comum na indústria de software.

Se você não tem um(a) colega disponível para revisar, tente ao menos “revisar você mesmo, com outro olhar”: termine o código, dê um tempo (algumas horas, um dia), depois releia simulando ser outra pessoa. Você pode se surpreender encontrando trechos confusos ou erros simples nessa segunda passada.

Manutenibilidade e dívida técnica: já falamos um pouco de dívida técnica – é o conceito de que atalhos tomados hoje podem virar boletos a pagar amanhã. Manutenibilidade é a característica de um sistema que indica o quão fácil é modificá-lo quando necessário. Se você aplica tudo que discutimos (modularidade, testes, documentação, versionamento etc.), está diretamente melhorando a manutenibilidade do seu projeto.

Por exemplo, um código modular e testado permite que você troque um módulo (digamos, experimentando um novo algoritmo de classificação) com a segurança de não quebrar os demais componentes. Já um código espaguete, sem testes, é frágil: qualquer mudança pode introduzir novos bugs sem que você perceba.

Em ciência de dados, a manutenção tem particularidades: imagine que semanas após entregar um modelo em produção, descobre-se que ele está enviesado e precisa ser ajustado. Se o pipeline de treinamento for bem organizado, você consegue inserir novas etapas (por exemplo, uma técnica de reamostragem ou inclusão de variáveis de fairness) sem refazer tudo do zero. Se for mal organizado, talvez seja mais fácil começar outro do zero – desperdiçando trabalho anterior. Isso acontece mais do que deveria: a equipe joga fora a primeira versão e faz uma “versão 2” do sistema corrigindo o design.

Quando isso ocorre, normalmente é porque a versão 1 adquiriu dívidas técnicas não pagas que inviabilizam a evolução. A dívida técnica oculta em sistemas de Machine Learning, conforme afirmam Sculley et al. (2015), manifesta-se em várias formas: dependências de dados instáveis, efeitos colaterais entre componentes, falta de monitoramento, entre outros.

Um exemplo concreto: você treina um modelo hoje com certos atributos e hiperparâmetros. Daqui a 6 meses, os dados mudaram e o modelo precisa ser re-treinado; porém, ninguém lembrou de versionar aquele conjunto de hiperparâmetros ou documentar passos específicos de pré-processamento e reproduzir o resultado se torna difícil – é como se o projeto tivesse “se perdido”. Essa dificuldade é evitável com disciplina e registro.

Pagar a dívida técnica significa alocar tempo para refatorar código quando necessário (melhorar a estrutura sem mudar a funcionalidade), eliminar “gambiaras”

temporárias substituindo por soluções perenes, adicionar testes que faltam em áreas críticas, remover código morto (features ou funções que não são mais usadas, que só poluem o sistema) e assim por diante. É comum gerentes e cientistas ficarem tentados(a) a sempre tocar para frente desenvolvendo novas funcionalidades ou modelos, mas ignorar essas tarefas de manutenção é pedir para o projeto colapsar adiante.

As metodologias ágeis até incluem isso na cadência: por exemplo, a cada sprint, reservar um percentual de tempo para dívida técnica e melhorias internas. Para resumir com um lema: “código é também um produto”. Após entregar o resultado de negócio (ex.: um modelo em produção), o código permanece e será reutilizado, ampliado – então, vale a pena mantê-lo saudável.

Por fim, vamos abordar um tópico mais técnico e igualmente vital: como escolher o estilo de código e as estruturas adequadas para escrever soluções eficientes e organizadas. No contexto de engenharia de software, discutem-se muito os paradigmas de programação, sendo dois muito conhecidos: programação estruturada (ou procedural) e programação orientada a objetos (POO). Cada paradigma oferece formas diferentes de pensar o código e na prática a maioria das linguagens e projetos permite misturar abordagens.

Programação estruturada/procedural é o estilo clássico de organizar o código em sequências de instruções, usando estruturas de controle (loops, condicionais) e, no máximo, agrupando lógicas em funções. Em Python, por exemplo, escrever um conjunto de funções no módulo principal e variáveis globais manipuladas por elas seria um estilo procedural.

Muitos projetos de ciência de dados são feitos assim: um script linear que lê os dados e depois executa uma série de passos, chamando algumas funções utilitárias no caminho. Não há nada de intrinsecamente errado nisso – para muitos casos simples, um script procedural basta. Contudo, à medida que o projeto cresce em complexidade, pode ficar difícil gerenciar o estado do programa apenas com variáveis globais e funções separadas.

```
# python
def calcular_media(lista):
    soma = 0
    contagem = 0
    for item in lista:
        if item is not None:
```

```
soma += item
contagem += 1
return soma / contagem

numeros = [10, None, 20, 30, None, 40]
print("Média:", calcular_media(numeros))
```

Código-fonte 14 – Exemplo em Python – Procedural
Fonte: Elaborado pelo autor (2025)

Programação Orientada a Objetos (POO): nesse paradigma, você cria classes, que são “modelos” para objetos que combinam dados (atributos) e comportamentos (métodos) relacionados. A ideia fundamental é encapsulamento: a classe encapsula um estado e expõe operações sobre esse estado.

Por exemplo, imagine que você queira representar todo o processo de treinamento e avaliação de um modelo em um só lugar. Você poderia criar uma classe `ModeloPredictivo` com atributos como `dados\_treino`, `dados\_teste`, `algoritmo` e métodos como `treinar()`, `avaliar()`.

A POO também suporta conceitos de herança e polimorfismo, que permitem criar hierarquias de classes especializadas. O benefício de usar objetos em ciência de dados aparece quando há múltiplas instâncias ou configurações simultâneas.

```
# python
class CalculadoraMedia:
    def __init__(self, lista):
        self.lista = lista

    def calcular(self):
        validos = [x for x in self.lista if x is not None]
        return sum(validos) / len(validos)

numeros = [10, None, 20, 30, None, 40]
calc = CalculadoraMedia(numeros)
print("Média:", calc.calcular())
```

Código-fonte 15 – Exemplo em Python – Orientado a Objetos
Fonte: Elaborado pelo autor (2025)

Já a programação funcional foca em funções puras, imutabilidade e composição. Em vez de alterar estados, você transforma dados por meio de funções. Paradigmas funcionais são comuns em linguagens como Haskell, mas o Python também oferece suporte parcial com funções como `map`, `filter`, `reduce`.

Esse estilo é útil quando se deseja evitar efeitos colaterais e tornar o código mais previsível e testável.

```
from functools import reduce

numeros = [10, None, 20, 30, None, 40]
validos = list(filter(lambda x: x is not None, numeros))
media = reduce(lambda acc, x: acc + x, validos) / len(validos)

print("Média:", media)
```

Código-fonte 16 – Exemplo em Python – Funcional
Fonte: Elaborado pelo autor (2025)

PARADIGMA	QUANDO USAR	VANTAGENS	DESVANTAGENS
Procedural	Scripts simples, protótipos rápidos	Fácil de entender e implementar	Difícil de escalar e manter
Orientado a Objetos	Projetos com múltiplas entidades e estados complexos	Modularidade, reutilização	Pode ser verboso e complexo
Funcional	Processamento de dados, pipelines, código declarativo	Concisão, menos efeitos colaterais	Pode ser menos intuitivo

Tabela 2 - Qual Paradigma Usar?
Fonte: Elaborado pelo autor (2025)

Cada paradigma tem seu lugar. Em projetos pequenos, scripts procedurais são rápidos e eficazes. À medida que a complexidade aumenta, a orientação a objetos ajuda a organizar melhor o código. Já a programação funcional brilha em

tarefas de transformação de dados e processamento paralelo. A escolha ideal depende do contexto, da equipe e da escalabilidade desejada. O importante é entender os trade-offs e aplicar o paradigma que melhor se adapta ao problema.

Além da organização em si, escolher as estruturas de dados adequadas é crucial para desempenho e uso eficiente de recursos (CPU, memória). Um(a) cientista de dados com boa prática de engenharia se preocupa, por exemplo, com a complexidade computacional de seu código. Se você tem de processar 1 milhão de registros, um algoritmo $O(n^2)$ (que escala com o quadrado do número de elementos) pode ser inviável – $1e6^2 = 1e12$ operações, o que pode levar dias ou anos em um computador comum.

Já um algoritmo $O(n \log n)$ ou $O(n)$ poderia lidar com 1e6 itens em tempos aceitáveis ($n \log n$ para 1e6 ~ 20e6, ou seja, 20 milhões de operações; $O(n)$ seria 1 milhão de operações). Esses cálculos aproximados nos fazem refletir sobre a necessidade de algoritmos eficientes. Às vezes, escrevemos código puramente pensando na lógica, sem considerar otimização – o que está ok em uma fase inicial –, mas ao levar para produção ou ampliar o volume, isso pesa.

Por exemplo, se você precisa verificar a existência de um item em uma coleção repetidamente, usar uma lista (em que a busca é $O(n)$ linear) pode tornar o código lento em um dataset grande; já usar um conjunto (set) ou dicionário (busca $O(1)$ amortizado, ou seja, constante) torna a verificação praticamente instantânea, não importando se há mil ou um milhão de elementos. Assim, conhecer estruturas básicas (listas, sets, dicts, filas, árvores, data frames etc.) e suas propriedades de desempenho é parte da caixa de ferramentas de um(a) cientista de dados engenheiro(a).

Ademais, pensar em uso de memória: em Python, trabalhar com objetos do Pandas ou listas gigantes consome RAM; se o dataset é enorme e não cabe em memória, estratégias como processar em stream (leituras parciais, usando iteradores ou geradores) podem ser necessárias. Linguagens de baixo nível ou recursos como numpy e pandas (que internamente usam código otimizado em C) ajudam aproveitando melhor CPU e também vencendo limitações do interpretador Python.

Por exemplo, multiplicar duas grandes matrizes element-wise usando loops Python puros seria incrivelmente lento, mas usando NumPy essa operação é feita em código C otimizado, explorando paralelismo de baixo nível – resultado: ordens de magnitude mais rápido. Portanto, um(a) bom(a) engenheiro(a) de software para ciência de dados conhece suas bibliotecas e as utiliza a favor: em vez de reinventar a roda com loops Python, muitas vezes há um método vetorizado ou uma função da biblioteca padrão que resolve com performance melhor. Isso libera tempo da CPU e do desenvolvedor ou desenvolvedora para focar no problema em si, não em micro-otimizações.

Falando em CPU e GPU: quando tratamos de modelos complexos (visão computacional, deep learning), o uso de GPU pode ser determinante. Saber quando migrar cálculo para GPU (e.g., usando PyTorch ou TensorFlow) é importante – mas também exige disciplina de software: o código precisa gerenciar adequadamente o dispositivo, transferência de memória etc. Um erro comum é copiar dados entre CPU e GPU em demasia, perdendo a vantagem. Ou seja, mesmo ao usar recursos avançados, engenharia conta.

Para quem lida com big data, a consideração sobe outro patamar: distribuição de processamento entre vários computadores. Ferramentas como Apache Spark, Dask ou Ray permitem distribuir tarefas de dados massivos. Contudo, para tirar proveito delas, o código deve ser escrito sem suposições de ambiente local, evitando estados globais não compartilháveis, e respeitando modelos de programação funcional (no Spark, por exemplo, as transformações são aplicadas em paralelo e variáveis definidas fora precisam ser enviadas aos executores cluster – requer cuidado). Assim, escalabilidade não é só questão de poder computacional, mas de design: código escalável tende a ser aquele que já é modular, separado em tarefas independentes e replicáveis.

Em síntese, um projeto de ciência de dados bem construído é tanto elegante quanto eficiente. Arquitetamos o software para ser compreensível e flexível, mas também olhamos para performance. Em muitos casos, clareza e eficiência andam juntas: por exemplo, substituir loops aninhados por operações de matriz não só acelera como muitas vezes simplifica o código (torna a intenção matemática mais explícita). Quando há trade-offs, pode-se iterar – primeiro garanta correção e clareza, depois, se necessário, otimize os gargalos identificados (profilers ajudam a

achar onde o código gasta mais tempo). A engenharia entra para sistematizar esse processo, em vez de depender de tentativa e erro caótica.

Para tornar isso concreto, vejamos um mini-estudo de caso: suponha que você tenha que implementar uma funcionalidade de detecção de anomalias em um dataset. Abordagem 1: você rapidamente escreve um código procedural que percorre os dados e marca outliers conforme alguma regra (ex.: valores além de 3 desvios-padrão). Funciona nos testes pequenos, mas quando você roda em um arquivo enorme, demora uma eternidade.

Ao investigar, você percebe que seu código faz diversos passes nos dados e várias concatenações de listas (operações custosas). Refatoração: você lembra que pode calcular média e desvio de forma vetorizada com numpy e fazer o filtro de outliers também com operações em lote. Você reescreve a solução usando numpy/pandas – o código não apenas ficou mais curto e legível (porque expressa a intenção estatística diretamente), como também ficou dezenas de vezes mais rápido graças às otimizações internas. Esse exemplo ilustra como pensar em estruturas e ferramentas adequadas transforma o resultado.

Outro exemplo, agora de POO: imagine que seu pipeline de dados envolva várias etapas condicionalmente aplicadas (talvez para cada tipo de variável, você faz algo diferente). Uma estrutura orientada a objetos, onde você tem uma classe abstrata `EtaapaPipeline` e subclasses como `EtaapaNormalizar`, `EtaapaCodificarCategorico`, cada uma implementando um método `executar(dados)` pode trazer duas vantagens: organizar o código por tipo de operação (alta coesão em cada classe) e permitir facilmente reordenar/compor etapas, talvez armazenando todas em uma lista `pipeline = [EtaapaNormalizar(), EtaapaCodificarCategorico(), ...]` e depois fazendo um loop genérico `for etapa in pipeline: dados = etapa.executar(dados)`.

Isso é flexível – para remover ou adicionar novas transformações, basta alterar a composição dessa lista, sem mexer em lógica de controle, e novas etapas podem ser criadas herdando da mesma interface. Em contrapartida, se tudo estivesse em um único giant script com condicionais, seria bem mais trabalhoso modificar a sequência de processamento ou reutilizar parte dela em outro contexto.

Em conclusão, reforçamos: engenharia de software aplicada à ciência de dados não tira a liberdade do(a) cientista; ela fornece ferramentas mentais e práticas para elevar a qualidade do trabalho. Ao adotar princípios de design, você escreve código modular e claro; ao utilizar boas práticas (versão, testes etc.), você ganha confiança e colaboração; ao entender estruturas e paradigmas, você produz soluções mais eficazes e escaláveis.

O resultado é que seus projetos terão maior longevidade e impacto – aquele modelo brilhante não ficará preso em uma planilha ou notebook esquecido, mas sim integrará sistemas, será atualizado com novos dados e realmente entregará valor continuamente.

Caso você queira se aprofundar ainda mais nesses tópicos, recomendo as seguintes ações:

(1) Reassistir às videoaulas complementares desta disciplina, em que discutimos ferramentas como Git, ambientes virtualizados e exemplos práticos de refatoração de código de data science.

(2) Realizar as atividades propostas no final desta aula, que envolvem identificar pontos fracos em pedaços de código e sugerir melhorias baseadas nos princípios que aprendemos;

(3) Revisitar materiais clássicos de engenharia de software (há muitos recursos abertos na web e no YouTube sobre design patterns, escrita de testes etc. aplicados em Python e R, por exemplo) – você ficará surpreso(a) em como muita coisa se traduz diretamente para o cotidiano de ciência de dados. E claro, pratique em seus próprios projetos de forma incremental: a cada novo código, tente aplicar pelo menos uma melhoria de engenharia. Com o tempo, isso se torna parte natural do seu modo de trabalhar.

MERCADO, CASES E TENDÊNCIAS

A integração entre engenharia de software e ciência de dados já é uma exigência do mercado, não só um tema acadêmico. Práticas sólidas de engenharia são decisivas para o sucesso ou fracasso de projetos de dados.

Case de fracasso: em 2020, o governo do Reino Unido perdeu quase 16 mil registros de COVID-19 por usar Excel além do limite técnico, sem planejamento de engenharia. A lição: improvisos podem gerar riscos graves; sistemas de dados exigem tecnologia adequada, controle de erros e testes.

Tendência de sucesso: empresas como Netflix e Uber investem em plataformas de MLOps (ex.: Metaflow), que facilitam reprodutibilidade, versionamento e automação de pipelines, tornando projetos escaláveis e confiáveis. Dominar ferramentas desse tipo é diferencial no mercado.

Educação e publicações: pesquisas mostram que código mal estruturado é comum e causa falhas. Universidades e cursos online estão enfatizando boas práticas de codificação e engenharia em ciência de dados (ex.: Harvard Data Science Review, Udacity, Coursera).

Futuro: cresce a oferta de frameworks e ferramentas automatizadas para projetos de dados (ex.: Cookiecutter Data Science, ferramentas de DataOps e observabilidade), além da especialização de papéis (engenheiro(a) de dados, cientista de dados, etc.).

Comunidade e recursos abertos: plataformas como MLOps Community, blogs (Towards Data Science) e projetos open-source oferecem conteúdo gratuito e atualizado sobre melhores práticas.

O mercado exige profissionais que combinem habilidades de programação, estatística e engenharia. Aproveite eventos, ferramentas gratuitas e busque aplicar boas práticas desde já — assim, você estará preparado(a) para atuar de forma profissional e evitar erros críticos no “mundo real”.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula destacamos a importância das práticas de engenharia de software para o sucesso em projetos de ciência de dados, mostrando que a maioria dos fracassos ocorre pela falta de código reproduzível e sustentável. Métodos iterativos e ágeis, baseados em feedback contínuo, foram apontados como mais adequados do que modelos tradicionais, pois oferecem flexibilidade para lidar com o caráter experimental desses projetos.

Foram abordados princípios fundamentais de design de código, como modularidade, coesão e baixo acoplamento, além de práticas essenciais como controle de versão, documentação, testes, revisões de código e gestão da dívida técnica. Também discutimos aspectos mais técnicos, incluindo paradigmas de programação, escolha de estruturas de dados e algoritmos, uso de bibliotecas otimizadas e estratégias para escalabilidade, paralelismo e confiabilidade do código na prática.

Por meio de exemplos do mercado, ficou evidente o impacto real das boas e más práticas de engenharia, reforçando que escrever código de qualidade é parte central do trabalho do cientista de dados.

Nos vemos na próxima aula. Até lá, bons códigos! Lembre-se: seja um(a) cientista de dados que também pensa e age como engenheiro(a) – seus futuros “eus” (e colegas) vão agradecer!

REFERÊNCIAS

KELION, L. **Excel**: Why using Microsoft's tool caused Covid-19 results to be lost. 2020. Disponível em: <https://www.bbc.co.uk/news/technology-54423988>. Acesso em: 11 dez. 2025.

LIFE CYCLE. **What is TDSP?** 2025. Disponível em: <https://www.datascience-pm.com/tdsp/>. Acesso em: 11 dez. 2025.

MASOLO, C. **Netflix enhances Metaflow with new configuration capabilities**. 2025. Disponível em: <https://www.infoq.com/news/2025/01/netflix-metaflow-configuration/>. Acesso em: 11 dez. 2025.

MUNSHI, A. **Why 80% of Data Science Projects Fail — And How to Fix It**. 2025. Disponível em: <https://www.linkedin.com/pulse/why-80-data-science-projects-failand-how-fix-arnav-munshi-d8pkc>. Acesso em: 11 dez. 2025.

NELSON, C. **Software Engineering for Data Scientists**: From Notebooks to Scalable Systems. Sebastopol: O'Reilly Media, 2024.

PRUIM, R.; GÎRJĂU, M.; HORTON, N. J. Fostering Better Coding Practices for Data Scientists. **Harvard Data Science Review**, v. 5 n. 3, Summer 2023. Disponível em: <https://hdsr.mitpress.mit.edu/pub/8wsiqh1c>. Acesso em: 11 dez. 2025.

SCULLEY, D. et al. **Hidden Technical Debt in Machine Learning Systems**. 2015. Disponível em: https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf. Acesso em: 11 dez. 2025.

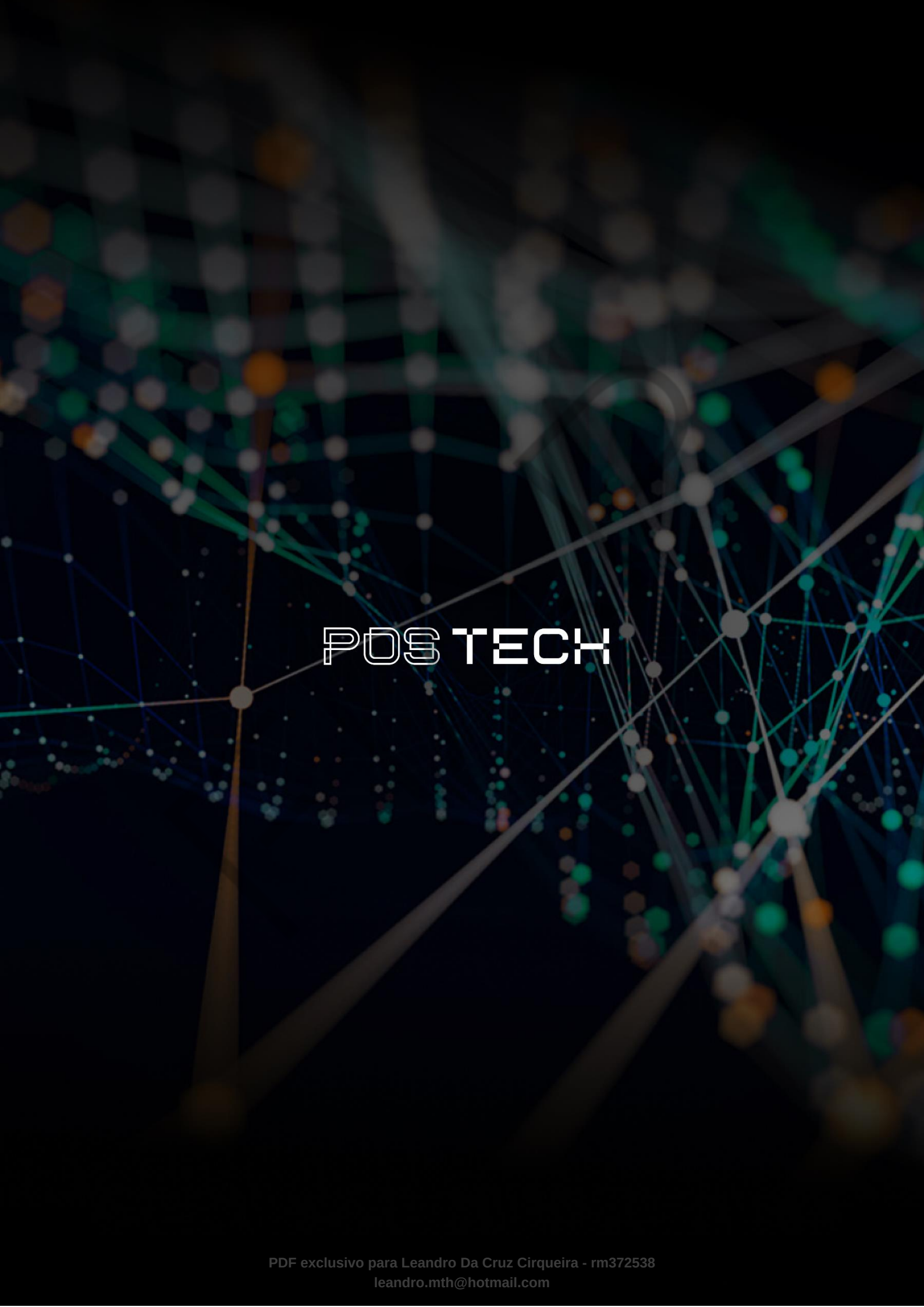
WIKIPEDIA COMMONS. **File:CRISP-DM Process Diagram.png**. 2012. Disponível em: https://commons.wikimedia.org/wiki/File:CRISP-DM_Process_Diagram.png. Acesso em: 11 dez. 2025.

WILSON, B. **Machine Learning Engineering in Action**. Shelter Island: Manning Publications, 2022.

PALAVRAS-CHAVE

Reprodutibilidade. Escalabilidade. Manutenibilidade.

ENAMP



POSTECH